

Python : fiche de synthèse

1. Règles de nommage et gestion des espacements

Python suit la convention **snake_case** : mots en minuscules séparés par des caractères *underscores* (« souligné du 8 »).

```
ma_variable = 42
def calculer_moyenne(liste): ...
```

- Constantes : `MAJUSCULES_AVEC_UNDERSCORES`.
- Classes : `PascalCase` (variante du `CamelCase` débutant par une majuscule).
- **Éviter les noms courts** sauf dans les boucles (`i`, `j`). Préférer des noms plus *explicites* (`for pos in range(len(lst))`).

Toute portion de code associée à une *structure de contrôle* nécessite une **indentation** : un décalage horizontal du code de **4 espaces**, par convention.

2. Variables et types

2.1. Généralités

Une **variable** est une *zone précise* de la mémoire vive de l'ordinateur (on parle *d'adresse mémoire*). À partir de cet emplacement, on trouvera une succession de *bits* dont l'agencement spécifique déterminera à la fois une **valeur** et un **type**.

Pour retrouver facilement et de manière fiable une zone précise de la RAM, on lui associe une étiquette : un texte, nommé **nom de la variable**. Par commodité, on assimile une variable et son nom. Dire « la variable `var` vaut `42` et est de type `int` » est donc un abus de langage fréquent.

Une variable peut être *immuable* ou *mutable*.

- **Variable immuable** : la « case mémoire » ne change pas de valeur. Quand on force la valeur à changer, une *nouvelle* case mémoire est utilisée, et le nom de la variable lui est associé (on perd le lien avec la case mémoire initiale). Python maintient un *identificateur numérique* pour chaque objet qu'on lui déclare : on l'obtient avec la commande `id()`. Il change lorsqu'on modifie la valeur d'une variable immuable.

Exemple : les int, float et str sont immuables ▼

```
a = 12.5
id(a)      # Renvoie par exemple 134256423983280
a += 1     # a désigne désormais la valeur 13.5
id(a)      # Renvoie un autre identifiant, 134256424593712 par exemple
```

- **Variable mutable** : la variable désigne toujours la *même* case mémoire, mais la ou les valeurs associées peuvent changer. À l'exception notable du **tuple**, les variables mutables sont souvent des « objets composés » : `list`, `dict`, objets définis par le programmeur dans un contexte de Programmation Orientée Objet (POO)...

Exemple : mutabilité et points de vigilance ⚠ ▼

Les types **mutables** peuvent être modifiés après création, deux variables peuvent pointer vers le **même objet**. C'est un piège classique !

```
a = [1, 2, 3]
b = a                # b N'est PAS une COPIE de a !
b.append(4)
print(a)             # [1, 2, 3, 4] ← a est modifiée !
```

Ici, `b` n'est qu'une autre étiquette qui, tout comme `a`, est « attachée » à la même zone mémoire.

Les types **immuables** (`int`, `str`, `tuple`) ne posent pas ce problème : toute « modification » crée un nouvel objet.

2.2. Types de base

Type	Mutable ?	Exemple
<code>int</code>	Non	<code>42</code>
<code>float</code>	Non	<code>3.14</code>
<code>str</code>	Non	<code>"Hello world !"</code>
<code>bool</code>	Non	<code>True, False</code>
<code>list</code>	Oui	<code>[1, 2, 3]</code>
<code>dict</code>	Oui	<code>{"a": 1}</code>
<code>tuple</code>	Non	<code>(1, 2, 3)</code>

Remarque : *f-string*? ▼

C'est une chaîne de caractères un peu particulière :

- elle est préfixée d'un `f` ;
- tout texte placé entre accolades sera remplacé par la valeur de la variable correspondante, qui **doit** pré-exister.

Exemple :

```
var = 42
msg = f"Réponse : {var}"      # La chaîne est créée à ce moment, avec la valeur actuelle de var
var = 43
print(msg)                   # Affichera « Réponse : 42 »
```

Attention, cette chaîne est *immuable*. Elle ne sera pas modifiée pour refléter les évolutions éventuelles de `var`.

2.3. Transtypage

C'est l'action de changer la structure interne d'un objet Python pour la faire correspondre à celle d'un autre type (en anglais on parle de *casting*). Il faut que cela soit permis, naturellement :

- un `int` peut toujours être changé en `float` avec la fonction `float()` ;

- un `float` peut toujours être changé en `int` avec la fonction `int()` (une *troncature* est réalisée : la partie décimale est supprimée) ;
- une `str` peut être convertie en `list` ou en `tuple` : `list("abc")` donne `['a', 'b', 'c']` ;
- une `str` peut *parfois* être convertie en `int` ou en `float` avec les fonctions correspondantes. C'est nécessaire lorsqu'on veut appliquer un traitement numérique au résultat d'une saisie utilisateur obtenue *via* la commande `input(...)`.
- une `list` (ou un `tuple`) peut être convertie en `str`, *mais PAS* au sens d'une opération réciproque de la précédente ! En effet, `str(['a', 'b', 'c'])` donne `"['a', 'b', 'c']"` ...
- on peut convertir beaucoup d'objets en booléen avec la fonction `bool`. La valeur `False` sera obtenue à partir de `0` ou `0.0`, `()` (tuple vide), `[]` (liste vide), `{}` (dictionnaire vide) ou `None`. Les autres valeurs seront transtypées en `True`.

 **Remarque : opération réciproque de `list("abc")`** 🤔 ? ▼

Pour recomposer `"abc"` à partir de `['a', 'b', 'c']`, il faut faire `"".join(['a', 'b', 'c'])`.

Plus généralement, `sep.join(liste)` concatène les éléments d'une liste de chaînes en les séparant par `sep` :

```
"-".join(["un", "deux", "trois"])    # "un-deux-trois"
" ".join(["Bonjour", "monde"])      # "Bonjour monde"
```

Attention ! Tous les éléments de la liste doivent être des `str`, sinon une `TypeError` est levée.

2.4. Affectations multiples

```
a, b = 1, 2                # Affectation parallèle
a, b = [1, 2]              # « Unpacking » depuis une liste
a, b = b, a                # Échange de valeurs (swap)
```

 **Remarque : *unpacking* étendu** ▼

Pour la culture (hors programme) : l'*unpacking* est l'action de « désempaqueter » un objet composé. Ainsi, lorsqu'on écrit `x, y = [1, 2]`, Python comprend qu'on souhaite affecter le 1^{er} élément de `[1, 2]` à `x` et le 2nd à `y`.

On peut aller plus loin : `x, *suite = [1, 2, 3, 4]` permet d'affecter le 1^{er} élément de la liste

à `x` et l'ensemble des éléments restants formera une nouvelle liste affectée à `suite`, grâce à la « syntaxe étoilée » `*suite`.

3. Opérateurs arithmétiques

3.1. Généralités

Opérateur	Rôle	Exemple
<code>+</code> , <code>-</code> , <code>*</code>	Addition, soustraction, multiplication	<code>3 * 4</code> → <code>12</code>
<code>/</code>	Division réelle	<code>7 / 2</code> → <code>3.5</code>
<code>//</code>	Division entière ou euclidienne (quotient)	<code>7 // 2</code> → <code>3</code>
<code>%</code>	Modulo (reste de la division entière)	<code>7 % 2</code> → <code>1</code>
<code>**</code>	Puissance	<code>2 ** 10</code> → <code>1024</code>

`//` et `%` sont complémentaires : `a == (a // b) * b + (a % b)`.

 Remarque : division euclidienne ▼

Cette dernière relation n'est rien d'autre qu'une réécriture en Python de la définition de la *division euclidienne*, vue en mathématiques au collège : le dividende `a` est égal au quotient `a // b` multiplié par le diviseur `b` augmenté du reste `a % b`.

Usages classiques :

- `n % 2 == 0` teste si `n` est pair ;
- `n % 10` extrait le chiffre des unités de `n` ;
- `n // 10` supprime le chiffre des unités de `n`.

 Remarque : `^` n'est pas la puissance ! ▼

En Python, `^` est l'opérateur **XOR bit à bit** (*bitwise exclusive or*), pas la puissance ! Ce piège vient d'autres contextes (calculatrices, Excel, mathématiques en ligne) où `^` signifie effectivement

« exposant ».

Le XOR compare deux entiers **bit par bit** : chaque bit du résultat vaut **1** si les deux bits en regard sont **différents**, **0** s'ils sont **identiques**.

```
# 2 en binaire : 0010
# 10 en binaire : 1010
# XOR bit à bit : 1000 → 8
2 ^ 10      # Donne 8, pas 1024 ! (XOR bit à bit de 2 et 10)
2 ** 10     # Donne 1024 (puissance en Python)
```

Le problème est *silencieux* : aucune erreur ici, Python calcule simplement autre chose que ce que l'on souhaitait le voir calculer !

3.2. Opérateurs d'affectation composée

Ils combinent une opération arithmétique et une affectation en une seule écriture :

```
x += 1      # équivalent à : x = x + 1
x -= 3      # équivalent à : x = x - 3
x *= 2      # équivalent à : x = x * 2
x /= 4      # équivalent à : x = x / 4
x //= 2     # équivalent à : x = x // 2
x %= 10     # équivalent à : x = x % 10
x **= 3     # équivalent à : x = x ** 3
```

C'est une écriture plus concise, très courante en Python. Elle ne change pas le comportement : la valeur de droite est calculée d'abord, puis réaffectée à la variable de gauche.

4. Tests et structures conditionnelles

4.1. Tests et opérateurs de comparaison

Un test est le résultat d'une opération de comparaison : `==`, `!=`, `<`, `<=`, `>`, `>=`, `is`, `is not`, `in`, `not in`. Son résultat est un *booléen* (un objet Python de classe `bool`), qui vaut `True` ou `False`

 Exemple : ▼

```
>>> 1 is True
False
>>> 1 == True
True
```

4.2. Structures conditionnelles

Une structure conditionnelle est une *structure de contrôle* (elle permet d'influer sur l'ordre d'exécution des instructions). Elle repose sur un ou plusieurs tests pour arbitrer sur le code qui sera réellement exécuté.

```
if condition:
    ...
elif autre_condition:      # autre_condition indispensable quand on utilise elif
    ...
else:                      # PAS de condition ici (jamais après un else)
    ...
```

⚠ Piège

Ne pas confondre ces deux codes !

```
if test():
    faire_ceci()
faire_cela()
```

Si `test()` donne `True`, `faire_ceci()` est exécutée. La fonction `faire_cela()` sera **toujours** exécutée, *quoi qu'il arrive* avant !

```
if test():
    faire_ceci()
else:
    faire_cela()
```

Si `test()` donne `True`, `faire_ceci()` est exécutée **mais pas** `faire_cela()`. La fonction `faire_cela()` sera exécutée **seulement si** `test()` renvoie `False` !

4.3. Opérateurs logiques

Ils permettent de *combiner plusieurs* tests.

Opérateur	Sens
<code>and</code>	vrai si les deux conditions sont vraies
<code>or</code>	vrai si <i>au moins une</i> est vraie
<code>not</code>	inverse la condition

```
if x > 0 and x < 10:           # équivalent à : if 0 < x < 10:
    ...

if not est_vide(...):
    ...
```

5. Structure linéaire 1 : la liste Python (type list)

La liste est une structure où chaque élément est numéroté par une position numérique : un `int` compté à partir de `0`.

5.1. Création

Création directe

```
lst = [1, 2, 3]                # Création par peuplement direct
lst = []                       # Création d'une liste vide
lst = list()                   # Variante de création vide
```

Création par accumulation

```
lst = []
for i in range(5):
    lst.append(i)
```

Création en compréhension (plus idiomatique) :

```
lst = [i**2 for i in range(5)]
lst = [x for x in lst if x % 2 == 0] # Avec filtre
```

5.2. Accès et modification

```
lst[0] # Renvoie le premier élément
lst[-1] # Renvoie le dernier élément
lst[1] = 99 # Modification de la valeur en 2e position
```

5.3. Ajout et suppression

Opération	Méthode	Coût
Ajout en fin	<code>lst.append(x)</code>	$\mathcal{O}(1)$ (à peu près)
Ajout/liste en fin	<code>lst.extend([x, ...])</code>	$\mathcal{O}(k)$ où k désigne la longueur de <code>[x, ...]</code>
Suppression en fin	<code>lst.pop()</code>	$\mathcal{O}(1)$
Suppression à l'indice i	<code>lst.pop(i)</code>	$\mathcal{O}(n)$
Suppression par valeur	<code>lst.remove(x)</code>	$\mathcal{O}(n)$

Remarque : complexité et coût ▼

Coût et complexité sont deux notions connexes mais différentes.

- Le coût mesure la quantité de ressources (temps, espace mémoire) pour qu'une opération s'exécute.
- La complexité vise à comprendre la façon dont évolue le coût lorsque la taille des données augmente (on parle de *comportement asymptotique*).

En pratique, on assimile les deux au lycée. Quant à la notion, il suffit pour le moment de retenir que $\mathcal{O}(\dots)$ signifie approximativement « de l'ordre de ... », « proportionnel à ... ».

5.4. Autres fonctions ou méthodes utiles

```
lst.reverse()           # Renverse la liste (en place, renvoie None)
lst.sort()              # Tri croissant en place (renvoie None)
lst.sort(reverse=True)  # Tri décroissant en place (renvoie None)
len(lst)                # Renvoie la longueur de la liste
```

⚠️ piège `lst = lst.sort()` ⚠️ ▼

`sort()` et `reverse()` modifient la liste **en place** et renvoient `None`. L'erreur classique est d'écrire :

```
lst = lst.sort()          # lst vaut maintenant None !
```

La bonne écriture est simplement :

```
lst.sort()                # lst est triée, et elle est toujours accessible sous ce r
```

5.5. Copie

```
copie = lst[:]            # Copie superficielle par slicing (hors programme)
copie = [x for x in lst]  # Copie par compréhension (explicitement au programme)

from copy import deepcopy # Technique utile si la liste contient des objets muta
copie = deepcopy(lst)      # Copie profonde (pour la culture)
```

5.6. Slicing (pour la culture)

Cette notion n'est pas explicitement au programme, mais peut être rencontrée à l'occasion.

```
lst[1:4]                  # Renvoie une nouvelle liste comportant les éléments d
lst[::2]                  # Nouvelle liste contenant 1 élément sur 2 de la liste
lst[::-1]                 # Renverse la liste (renvoie aussi une nouvelle liste)
```

5.7. Variante : le tuple

Le tuple est une liste **immuable** : on ne peut pas le modifier après création. Syntaxe : `(1, 2, 3)` ou `1, 2, 3`. Utilisé pour des collections fixes ou les valeurs de retour multiples.

 **Remarque : tuple immuable... mais pas toujours sans surprise 🤖 ! ▼**

Un `tuple` est immuable : on ne peut pas *remplacer* un de ses éléments. Mais s'il *contient* des objets mutables (comme des listes), ceux-ci restent modifiables :

```
t = (0, [1, 2])
t[1].append(3)
print(t)           # Affiche (0, [1, 2, 3]) ← le tuple n'a pas changé, mais son contenu l'a
```

L'immuabilité du tuple garantit que ses *références* ne changent pas, pas que les objets référencés sont eux-mêmes figés.

6. Structure linéaire 2 : le dictionnaire (type dict)

Le dictionnaire est une structure où chaque valeur est associée à une clef, un peu comme si elle était étiquetée. Les clefs doivent être de types *immuables*.

6.1. Création

```
dic = {}           # Création d'un dictionnaire vide
dic = dict()       # Variante
dic = {"nom": "Alice", "âge": 30}  # Création d'un dictionnaire déjà peuplé
```

6.2. Accès, modification, ajout, suppression, appartenance

```
dic[clef]          # Accès à une valeur connaissant la clef (KeyError si
dic[clef] = valeur  # Modification OU **ajout** d'une valeur associée à un
del dic["nom"]      # Suppression d'un couple clef-valeur
```

L'appartenance peut porter sur deux domaines : les clefs ou les valeurs.

```
key in dic          # Renvoie True ou False, selon que key est ou non une
val in dic.values() # Renvoie True ou False, selon que val est ou non une
```

6.3. ⚠ dict.get(clef) vs dict[clef]

La méthode `get` des dictionnaires devient de plus en plus populaire, au bac...

Elle trouve son intérêt lorsqu'on veut extraire de l'information d'un dictionnaire sans être sûr au préalable que la clef existe bien.

Supposons une variable `clef_absente` définie (de type immuable), et un dictionnaire `dic` qui **ne** contient **pas** cette clef.

```
dic[clef_absente]          # Provoque une exception KeyError
dic.get(clef_absente)      # Renvoie None pour une clef absente du dictionnaire
dic.get(clef_absente, 42)  # Renvoie 42 pour une clef absente du dictionnaire
```

Il faut privilégier `get()` lorsqu'on n'est pas sûr que la clef existe. Le code est plus lisible que `if key in dic: ...` (mais moins explicite).

7. Définir une fonction

7.1. Généralités

Une fonction est une structure de contrôle : elle influe sur l'ordre dans lequel des instructions sont exécutées. C'est au minimum un moyen de grouper en les nommant plusieurs instructions : cela facilite leur réemploi, et simplifie la maintenance du code (évolutions, corrections de bogues...). Une fonction peut (ou non) avoir des paramètres (des valeurs d'entrée) et 0, une ou plusieurs valeurs de retour (en sortie).

```
def aire_rectangle(largeur: float, hauteur: float) -> float:
    """Calcule et retourne l'aire d'un rectangle.

    largeur : largeur en unités
    hauteur : hauteur en unités
    """
    return largeur * hauteur
```

- La **docstring** (chaîne de documentation, entre `"""`) documente la fonction. Elle est censée indiquer son action, lister ses paramètres et valeur(s) de retour (noms et types).
- Les **annotations de type** (`: float`, `-> float`) sont indicatives, non contraignantes. Elles aident à mieux percevoir la nature de « ce qui entre et ce qui sort » d'une fonction.

7.2. Argument avec valeur par défaut

```
def saluer(nom: str, message: str = "Bonjour") -> str:  
    return f"{message}, {nom} !"
```

7.3. ⚠ Piège : valeur par défaut mutable ⚠

Ne jamais utiliser un type mutable comme valeur par défaut :

```
# Ici, la MÊME liste est partagée entre tous les appels !  
def ajouter(x, lst=[]):  
    lst.append(x)  
    return lst  
  
ajouter(1)           # Renvoie [1]  
ajouter(2)           # Renvoie [1, 2], pas [2] !
```

La bonne approche est :

```
def ajouter(x, lst=None):  
    if lst is None:  
        lst = []  
    lst.append(x)  
    return lst
```

7.4. Valeurs de retour

Une fonction Python peut *ne pas* comporter le mot-clef `return` : dans ce cas, elle renvoie *implicitement* `None`.

Autrement, une expression ou un nom de variable doit suivre `return`. Le type de cette expression ou

variable est sans importance (`dict`, `list`, `str`...).

Exception : un `return` seul (non suivi) signifie `return None`. Certains l'emploient, en dehors de la dernière ligne d'une fonction, pour marquer explicitement un point de sortie « anticipé ».

Plusieurs valeurs peuvent être renvoyées en écrivant `return a, b, c` (par exemple) : c'est en réalité un `tuple` qui est alors renvoyé. On peut alors écrire `x, y, z = f()` pour « déballer » le tuple obtenu.

7.5. Portée des variables

Une variable définie **dans** une fonction est **locale** : elle n'existe que *pendant l'exécution* de cette fonction et est *inaccessible* depuis « l'extérieur » (de cette fonction).

Une variable définie **hors** de toute fonction est **globale** : elle est *lisible* depuis l'intérieur d'une fonction, mais on **ne** peut **pas** la réaffecter sans précaution.

```
x = 10          # variable globale

def f():
    print(x)    # lecture autorisée

def g():
    x = 99      # crée une variable LOCALE x, ne modifie pas le x global !
```

Remarque : global ▼

Le mot-clef `global` autorise la réaffectation d'une variable depuis une fonction, mais son usage est **fortement déconseillé** ! Il rend le code difficile à déboguer car il autorise des interactions complexes. Il faut absolument privilégier l'usage de paramètres et de valeurs de retour dans l'écriture de fonctions.

8. Boucles

8.1. Boucle bornée : for

On emploie ce genre de boucle lorsqu'on peut connaître à l'avance le nombre répétitions à effectuer.

```
for i in range(n):           # i prendra successivement les valeurs 0, 1, ..., n-1
for i in range(a, b):        # i prendra successivement les valeurs a, a+1, ..., b-1
for i in range(a, b, pas):    # Idem mais avec un pas (écart ou saut, qui peut être
```

8.2. Boucle non bornée : while

On emploie ce genre de boucle lorsqu'on **NE** peut **PAS** connaître à l'avance le nombre répétitions à effectuer.

```
i = 0
while i < len(lst):          # attention : < et non <= si on veut éviter l'IndexError
    ...
    i += 1                   # ne pas oublier de faire avancer l'indice !
```

Les mêmes opérateurs logiques (`and`, `or`, `not`) permettent de combiner plusieurs conditions dans le test du `while`.

Instructions complémentaires :

- **break** : interrompt immédiatement la boucle.
- **continue** : passe directement à l'itération suivante.

8.3. Parcourir une liste, un tuple ou une chaîne

La façon de parcourir ces trois types composés est identique. Elle peut se faire de 3 manières différentes.

Parcours par position : un indice décrit tous les numéros d'emplacements possibles (comptés à partir de 0) :

```
for i in range(len(lst)):
    print(lst[i])
```

Parcours par valeur : on récupère séquentiellement chaque élément. Dans certains autres langages, le mot-clef spécifique `foreach` est utilisé (ce qui est plus explicite).

```
for elt in lst:
    print(elt)
```

Parcours mixte : par position ET par valeur :

```
for pos, val in enumerate(lst):
    print(pos, "->", val)
```

Les mêmes formes s'appliquent aux **tuples** et aux **chaînes** (`str`).

8.4. Parcourir un dictionnaire

On retrouve les trois types de parcours précédents : parcours par clef (équivalent au parcours par position), par valeur et mixte.

Parcours par clef :

```
for clef in d:
    ...
```

Équivalent à : `for clef in d.keys()`

Parcours par valeur :

```
for val in d.values():
    ...
```

Parcours mixte :

```
for clef, val in d.items():
    ...
```

S'il n'y en a qu'un à retenir, c'est lui !

9. Fonctions intégrées utiles

Fonction	Rôle	Exemple
<code>len(x)</code>	Longueur d'une séquence	<code>len([1, 2, 3]) → 3</code>
<code>int(x)</code>	Conversion en entier	<code>int("42") → 42</code>
<code>ord(c)</code>	Code Unicode d'un caractère	<code>ord('A') → 65</code>
<code>chr(n)</code>	Caractère depuis un code Unicode	<code>chr(65) → 'A'</code>
<code>bin(n)</code>	Représentation binaire	<code>bin(10) → '0b1010'</code>
<code>hex(n)</code>	Représentation hexadécimale	<code>hex(255) → '0xff'</code>

10. Modularité et modules utiles

10.1. Intérêt

La **modularité** consiste à organiser le code en fichiers (modules) réutilisables. Importer du code peut se faire de trois manières différentes :

- **simple mais souvent excessive** : `from math import *`. Les fonctions ou constantes ainsi importées sont utilisables nativement (exemple : `pi` ou `log2`).
- **ciblée** : `from random import random` importe la fonction `random()` du module `random` (variante avec *alias* : `from random import randrange as rr`, qui permet de « raccourcir » les noms). Les fonctions ou constantes ainsi importées sont utilisables nativement (exemple : `random` ou `rr`, qui sera synonyme de `randrange`). Toute autre fonction *non spécifiquement* importée sera inutilisable (pas moyen d'appeler `randint`, ainsi).
- **minimaliste** : `import math`. On peut alors accéder à *toutes* les fonctions du module en préfixant leurs noms par celui du module et en usant de la notation pointée (exemple : `math.sqrt`). Cette approche est très utile lorsque deux modules différents définissent le même nom de fonction : tout autre forme d'import masquerait l'une des deux !

10.2. Créer un module Python

De façon basique, rien n'est plus simple : n'importe quel fichier Python peut être importé. C'est

particulièrement facile lorsque les fichiers sont dans le même répertoire. Supposons que `A.py` et `B.py` le soient, on peut mettre dans `B.py` l'instruction `from A import *` : toute fonction (ou variable) déclarée dans `A.py` sera alors utilisable dans `B.py`.

10.3. `if __name__ == "__main__"`

Cette syntaxe idiomatique de Python permet de n'exécuter du code présent dans un fichier / module **que** si ce fichier / module est **directement exécuté**. Ce code **N'est PAS** exécuté lorsque ce fichier / module est importé. Cette approche est généralement utilisée pour inclure du code de test dans un module.

```
def ma_fonction():
    ...

if __name__ == "__main__":
    # Ce bloc n'est exécuté que si on lance ce fichier, pas s'il est importé depuis
    ma_fonction()
```

10.4. Module `random`

Module assurant la génération de nombres aléatoires.

```
import random

random.random()          # Génère un float aléatoire dans [0.0, 1.0[
random.randint(a, b)     # Génère un entier aléatoire dans [a, b] (bornes incluses)
random.randrange(a, b)   # Génère un entier aléatoire dans [a, b[ (borne haute excl)
```

La fonction `choice` permet de choisir au hasard un élément dans une liste :

```
random.choice([True, False])          # Simule un pile ou face (pièce)
random.choice(["Alice", "Bob", "Charlie", "Diana"]) # Tire un nom au sort dans une
```

10.5. Module `pprint`

Ce module permet un « meilleur affichage » en console. Il peut s'avérer utile pour « déboguer au `print` » un code impliquant des listes ou des dictionnaires (ce qui peut servir en Épreuve Pratique).

👉 C'est spécialement vrai lorsque listes ou dictionnaires sont imbriqués !

```
from pprint import pprint as pp

pp(donnees)           # Affichage lisible, hiérarchisé et structuré
pp(donnees, width=60) # Limite la largeur à 60 caractères (passage à la ligne au
```

11. Les assertions

Une **assertion** vérifie qu'une condition est vraie à un instant donné. Si elle est fausse, le programme provoque une `AssertionError`.

On utilise souvent les assertions pour vérifier des préconditions, ou (en NSI), pour valider le comportement d'un code.

Vérification d'une précondition	Vérification de la conformité d'une sortie
<pre>def inverse(x): assert x != 0, "0 n'a pas d'inverse" return 1/x</pre>	<pre>def double(n: int) -> int: return 2 * n assert double(3) == 6 assert double(0) == 0 assert double(-1) == -2</pre>

Bonnes pratiques

- Écrire les assertions **immédiatement après la définition** de la fonction, comme jeu de tests minimal.
- Tester les **cas limites** : liste vide, zéro, valeurs négatives, chaîne vide.
- Les assertions peuvent être désactivées à l'exécution avec le flag `-O` ; elles servent au **développement**, pas à la validation en production.

```
# Exemple avec cas limites
def premier_element(lst: list):
    assert len(lst) > 0, "La liste ne doit pas être vide"
    return lst[0]
```

Le message *facultatif* après la virgule précise la cause de l'erreur.